

GL2GPU Tint 三设备正确性与 Spark 性能复现实验

实验日期：2026-07-14 至 2026-07-15

最终实现提交：8ebfd051751f23a8fa79e43434e8444c9cdd9b0b

Spark 基线标签：sparkjs-0 (26e17df5488f4c46af6a86211f615034134f2971)

浏览器模式：三台设备均为 headed Chrome, deviceScaleFactor=1

1. 结论

本轮在一台 Apple M4 MacBook Air、一台 Apple M4 Mac mini 和一台 Apple M1 MacBook 上重新运行完整 WebGL CTS，并用同一份 Spark v2.1.0、三份相同哈希的 PLY 场景比较原生 WebGL 与 GL2GPU Tint。核心结论如下。

1. **三台设备的严格 CTS 汇总均为 2864/2864**。每台都是 15 份互不重叠的 headed Chrome 分块，唯一 test page 数为 2864，失败、timeout、重复和缺口均为 0。三机使用相同 CTS commit、相同 GL2GPU bundle、相同 harness 和相同审计补丁。
2. **runtime 修复与 CTS fixture 修正必须分开理解**。GL2GPU 将不安全的 boolean uniform specialization 从默认开启改为显式 opt-in，修复了 4 个 uniform/bool 语义回归。Pinned CTS snapshot 另有两个彼此矛盾的 identifier fixture；最终 2864/2864 使用了只修改测试输入的审计补丁，补丁 SHA-256 已固定并写入每个结果。
3. **Spark steady-state GPU throughput 在三机全部快于原生 WebGL**。Van Gogh Room 为 1.076x–1.272x，cleaned bicycle 为 1.801x–2.098x，full bicycle 为 2.768x–3.154x。方向在 M1 与两台 M4 上一致。
4. **加速没有以错误画面换取**。九组 Tint/WebGL 对比的最差 RMSE 为 0.0028562，最低 PSNR 为 50.884 dB，最低 SSIM 为 0.9999133。两台远程机器的代表图已复制回本机逐张目检，没有黑屏、Y 翻转、裁切、丢几何或透明混合异常。
5. **Spark 加速不能归因于 Tint 编译器本身**。实验显式启用了 GL2GPU 的静态相机 splat vertex precompute/compaction：一次 compute pass 预计算每个 splat 的 clip center、屏幕轴、颜色与有效标准差，剔除不可见实例，再把结果作为 instance vertex buffer 输入简化后的 render shader。它减少了每帧顶点工作量，但增加了加载阶段成本。
6. **通用 Tint/manual 性能仍有缺口**。2026-07-14 的 GL2GPU Demo 对照中，Tint 在 Aquarium、MotionMark、Sprites 上只有 manual 的 0.666x–0.861x (100k) 和 0.667x–0.800x (300k)。因此准确结论是“当前 Spark 静态场景路径显著加速”，不是“所有 Tint shader 都比手写 shader 快”。

2. 版本与可复现性

2.1 固定版本

组件	版本或 SHA
GL2GPU 最终实现	8ebfd051751f23a8fa79e43434e8444c9cdd9b0b
Spark precompute 基线 tag	sparkjs-0 / 26e17df5488f4c46af6a86211f615034134f2971
GL2GPU release bundle SHA-256	8977f67c2856c1f2f4925dc5dd2f77548ed6b4d5cf0e92703a993eabcd58b498
WebGL CTS harness SHA-256	407a4994bcd3514acec8b9febcb4b1543ee1da42bfa049d93052baeb08838e1b
Spark harness SHA-256	24929f9087593322107c1dc3f193bfac89a7c4abffcba09880ab9021618587cf
Khronos WebGL CTS	064aaf18207438d4f6dd10c98b02b25778257b7f
CTS fixture patch SHA-256	846c6bea3331eb426d4c706e07734a16f560d07adf837369ddcb111f8002ca56
Spark	v2.1.0 / f22236f95fdd8078f0c12e3aab479523d401daf6

实验在实现 commit 创建前运行，因此原始 JSON 的 `gl2gpuCommit` 字段记录的是当时的 基线 `26e17df`。实验后没有再修改受测 runtime 与 harness，而是将同一棵实现树提交为 `8ebfd05`。上表的 bundle 和 harness SHA-256 与三机原始结果逐字节一致，是比 dirty worktree 的 commit 字段更强的实验身份凭据。

2.2 设备

设备	SoC / GPU	内存	macOS	Chrome
Echo M4	Apple M4, 10-core GPU	32 GiB	26.3.1 (25D771280a)	150.0.7871.115
Mac mini M4	Apple M4, 10-core GPU	16 GiB	26.2 (25C56)	150.0.7871.124
Genesis M1	Apple M1, 8-core GPU	16 GiB	26.2 (25C56)	150.0.7871.102

三台机器都连接在线桌面会话并启动可见 Chrome 窗口，不使用 headless Chrome。Chrome 主版本一致，但 patch version 不完全一致，因此小于几个百分点的跨机差异不应解释成 硬件因果关系。

3. WebGL CTS

3.1 Runtime 修复

先前的完整三机运行均为 `2858/2864`，六个失败集合完全相同。其中四个用例涉及 boolean uniform 的默认值、跨 program 状态和 bool/int/float cast：

- `conformance/glsl/bugs/bool-type-cast-bug-int-float.html`
- `conformance/uniforms/uniform-default-values.html`
- `conformance/uniforms/uniform-values-per-program.html`
- `conformance2/glsl13/bool-type-cast-bug-uint-ivec-uvec.html`

根因不是 Tint 的 GLSL 解析，而是 GL2GPU 在 link 后把选定的 boolean uniform lower 成 WGSL pipeline override。这个优化默认开启时，把 WebGL 的“每个 program 拥有独立、可变且有规范默认值的 uniform 状态”压缩成 pipeline variant 状态，导致 默认值和 program 切换语义不再可靠。`8ebfd05` 将该优化改成只有 `__HYD_STATIC_BOOLEAN_UNIFORM_VARIANTS === true` 时才启用。生产默认路径保留普通 uniform 读取；上述四个定向用例随即恢复通过。

3.2 两个 CTS snapshot fixture 问题

剩余两个失败不是应由 GL2GPU 迎合的 WebGL 行为：

Fixture	snapshot 中的问题	审计补丁
shader-with-double-underscore.html	用来验证双下划线 identifier 可编译，但 fragment shader 写了 vertex-only 的 attribute	改成合法的 vertex attribute + vertex/fragment varying，保留双下划线测试目标
shader-with-reserved-words.html	同一 snapshot 的规范已允许 identifier 中出现 __，旧 future-word 列表仍要求 __foo、foo__bar 编译失败	从旧列表移除这两个条目；gl_、webgl_ 规则保持不变

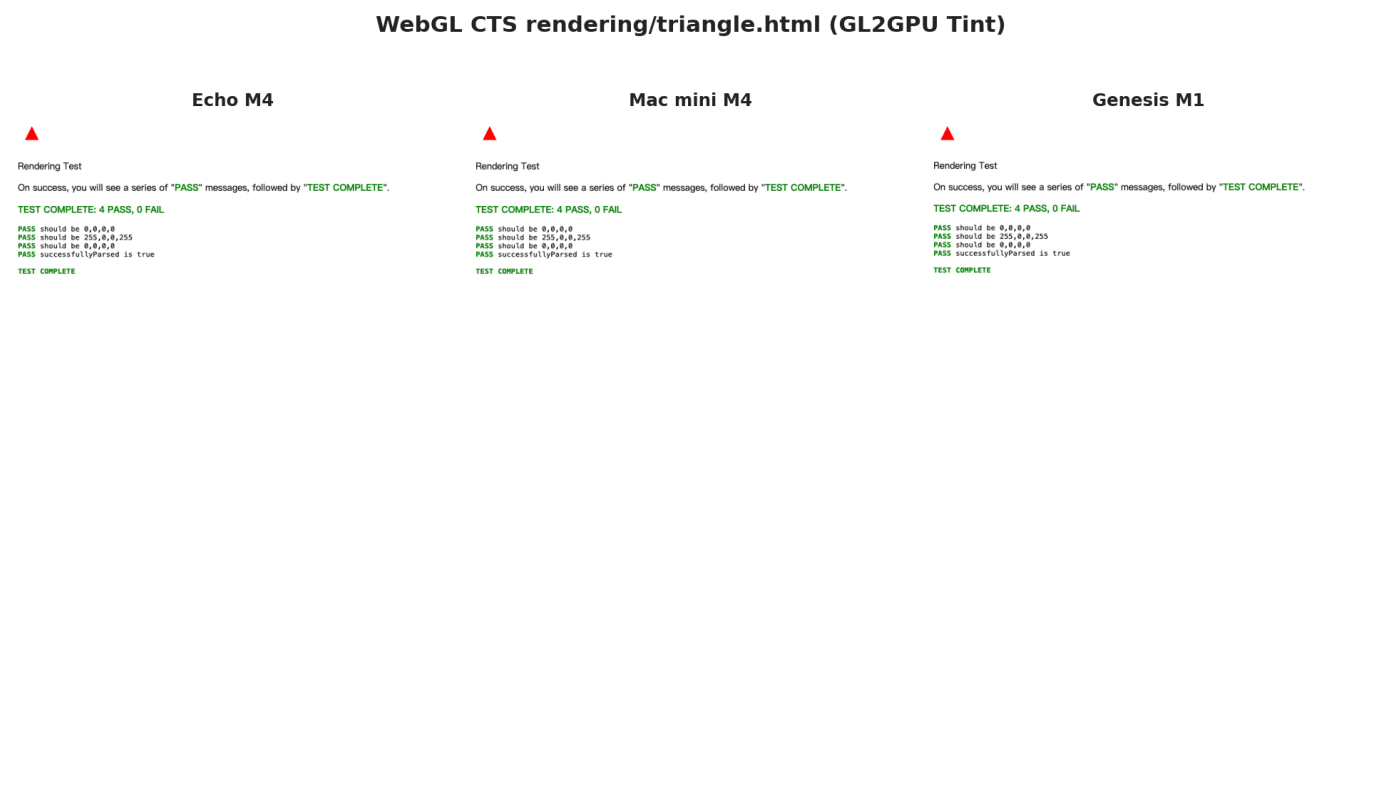
补丁只作用于 /tmp 或远程 CTS checkout，不修改 GL2GPU runtime。Harness 默认拒绝 dirty CTS worktree；只有 diff SHA-256 精确等于审计值并显式传入 --allow-cts-upstream-fixes true 才允许运行。因此最终结果应准确表述为：

GL2GPU runtime 修复了四个真实语义回归；在 pinned CTS commit 加两处 fixture-only 一致性修正后，三机严格汇总均为 2864/2864。

3.3 最终汇总

设备	分块	唯一页面	通过	失败	Timeout	重复	完整
Echo M4	15	2864	2864	0	0	0	是
Mac mini M4	15	2864	2864	0	0	0	是
Genesis M1	15	2864	2864	0	0	0	是

三机最终汇总还验证 CTS commit、fixture diff SHA、bundle SHA 和 harness SHA 全部一致。下图是三台远程/本地 headed Chrome 对 rendering/triangle.html 的实际截图； 每张都显示红色三角形和 4 PASS, 0 FAIL 。



4. Spark v2.1.0 实验方法

4.1 场景与相机

Scene	文件大小	Camera	PLY SHA-256
Van Gogh Room	23,208,408 B	cameras_test.json[0]	3c52f6f...e7f5
Bicycle cleaned	263,648,100 B	cameras_test.json[0]	b62d9502...8641
Bicycle full	1,520,726,124 B	cameras_test.json[0]	64d357cb...227

三台机器都验证了完整 SHA-256。相机按 WebSplatler 的 OpenCV column convention 解释；canvas backing resolution 使用 camera 原始尺寸的 1/4。相机选择、viewport、场景文件和 Spark module 在 WebGL/Tint 两边完全相同。

4.2 测量协议

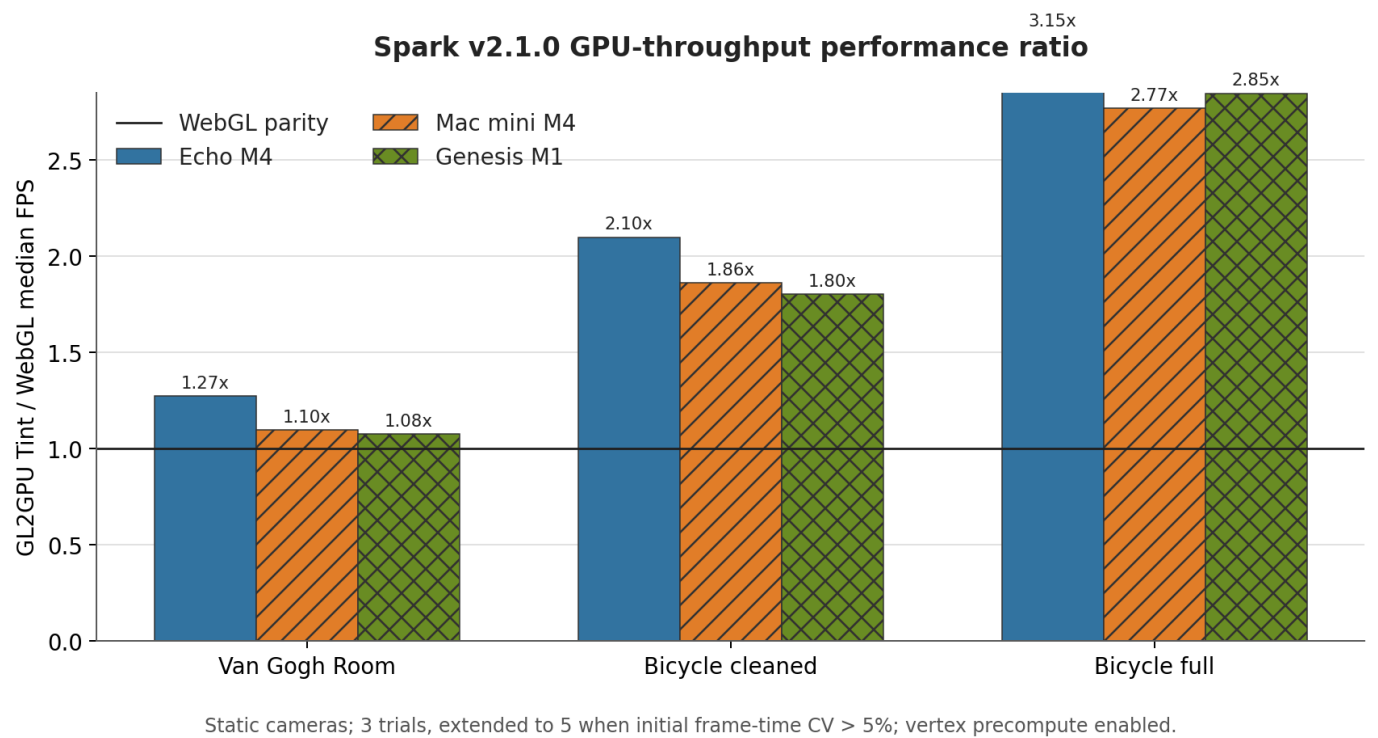
- 两种模式仅改变 context 路径：原生 Spark WebGL 与 GL2GPU Tint runtime translation。
- Tint 路径必须满足 shaderDbRequests=0、无 page error、无 WebGPU validation error、无 shader translation failure，并确认 vertex precompute hook 实际 applied。
- 每个 trial 先做 2 个 warmup batch，再做 9 个 measurement batch，每 batch 2 帧；指标等待 GPU 完成，不受显示器 60 Hz RAF 上限限制。
- 每个 scene/mode 先按 ABBA 顺序运行 3 次。若任一 mode 前三次 median frame time 的 coefficient of variation 大于 5%，两边都扩展至 5 次。
- 三台设备的 room 与 cleaned bicycle 都扩展到 5 次；full bicycle 前三次已稳定，按协议保留 3 次。表中使用有效 trial median 的 median。
- 画质对每对同相机截图计算 normalized RMSE、PSNR 和 SSIM；RMSE gate 为 0.02。

M4 首次 preflight 的页面已经完成渲染，但非登录 SSH PATH 中找不到已安装的 /opt/homebrew/bin/magick，截图分析返回 spawnSync magick ENOENT。该轮被严格标为 invalid 并全部丢弃；补齐 PATH 后从头重跑，以下只使用第二轮有效结果。

5. Spark 性能结果

5.1 Steady-state throughput

设备	Scene	Trials/模式	WebGL FPS	Tint FPS	Tint/WebGL	Frame time 降低	Tint/WebGL load
Echo M4	Van Gogh Room	5	106.952	136.008	1.272x	21.4%	3.76x
Echo M4	Bicycle cleaned	5	103.950	218.103	2.098x	52.3%	1.84x
Echo M4	Bicycle full	3	32.946	103.896	3.154x	68.3%	1.17x
Mac mini M4	Van Gogh Room	5	119.868	131.536	1.097x	8.9%	4.08x
Mac mini M4	Bicycle cleaned	5	105.597	196.367	1.860x	46.2%	1.92x
Mac mini M4	Bicycle full	3	33.239	92.017	2.768x	63.9%	1.12x
Genesis M1	Van Gogh Room	5	120.012	129.074	1.076x	7.0%	3.64x
Genesis M1	Bicycle cleaned	5	88.790	159.936	1.801x	44.5%	1.82x
Genesis M1	Bicycle full	3	25.947	73.842	2.846x	64.9%	1.16x



场景越大，加速越稳定且越明显。Full bicycle 的原生 WebGL median frame time 为 30.1–38.5 ms，Tint/precompute 为 9.6–13.5 ms。Van Gogh Room 的 GPU 工作量 较小，固定提交、同步与 vertex-buffer 成本占比更高，所以收益缩小到 1.076x–1.272x。

代价主要出现在加载阶段。Tint/WebGL median load–time ratio 为 room 的 3.64x–4.08x、cleaned bicycle 的 1.82x–1.92x、full bicycle 的 1.12x–1.17x。这符合一次性 WGSL translation、pipeline creation、compute precompute、readback visible index 和 compact buffer 建立的成本模型。静态观看时间越长，steady-state 收益越可能摊薄这部分开销；频繁变化的相机/数据则不能直接套用本表。

5.2 为什么会快

原生 Spark vertex shader 每帧、每个实例都要从 ordering/splat textures 读取数据， 计算 clip center、2D covariance eigenvectors、屏幕空间 axes、alpha/标准差和最终 quad offset。GL2GPU 的显式 precompute 路径做了三件事：

- 1. 把可复用的每实例计算搬到一次 compute dispatch， 写入四个 storage buffer；
- 2. 读取 adjusted-stddev 可见性结果， 生成 visible index 列表并在 GPU 上 compact；
- 3. render pipeline 只消费 compact 后的 clip/axes/RGBA/stddev instance attributes， 每个 quad vertex 仅完成少量插值与位置重建。

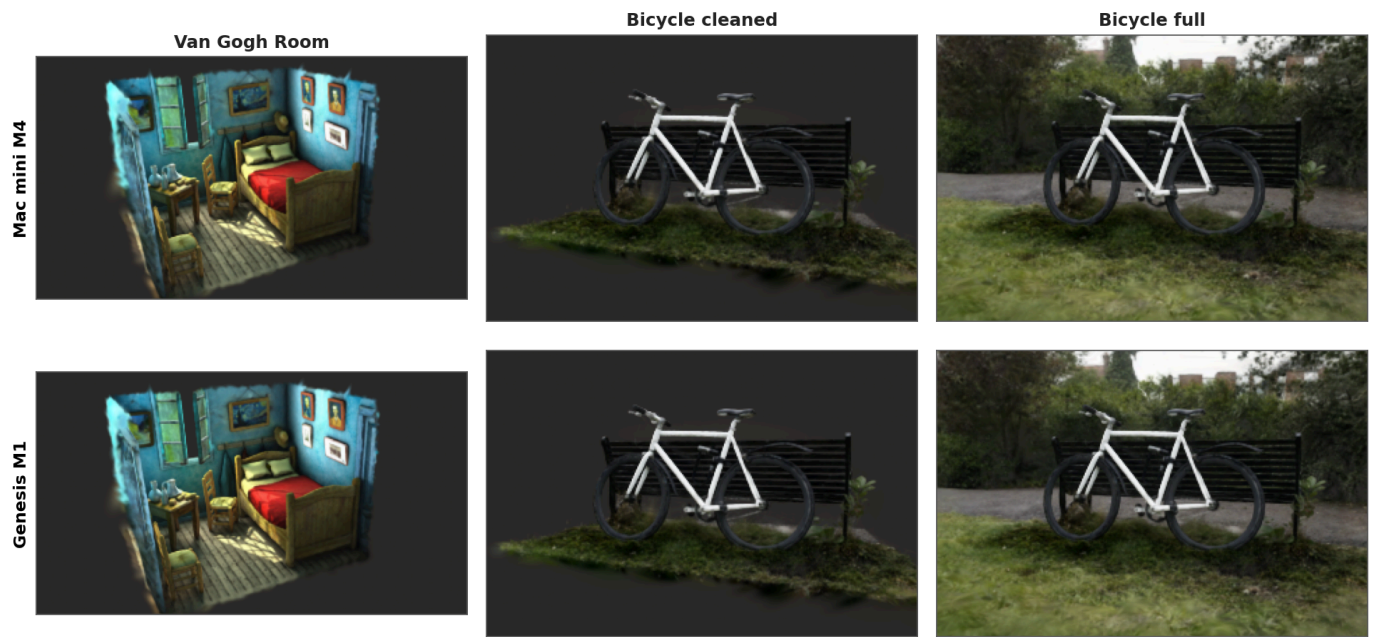
因此 full bicycle 同时减少了“参与 draw 的 instance 数”和“每个可见 instance 的 重复顶点计算”。该优化是由 Spark shader 结构触发并由 benchmark 显式调用的 GL2GPU 能力， 不是 Tint WASM 对任意 GLSL 自动执行的通用 pass， 也不应作为 Dawn/Tint compiler speedup 的证据。

6. 画质与视觉检查

Scene	三机 RMSE 范围	PSNR 范围 (dB)	SSIM 范围
Van Gogh Room	0.0005660—0.0005671	64.926—64.944	0.9999967—0.9999967
Bicycle cleaned	0.0010225—0.0010227	59.805—59.807	0.9999607—0.9999607
Bicycle full	0.0028559—0.0028562	50.884—50.885	0.9999133—0.9999133

所有 RMSE 都远低于 0.02 gate。指标之外， 两台远程机器的 Tint 代表图和对应 WebGL 图均以 original resolution 逐张查看： 房间的床、墙面、窗户和地板完整； cleaned bicycle 保留暗背景与局部地面； full bicycle 的真实背景、车架、长椅和前景草地一致。 没有出现此前 texture-coordinate bug 所造成的 Y 翻转。

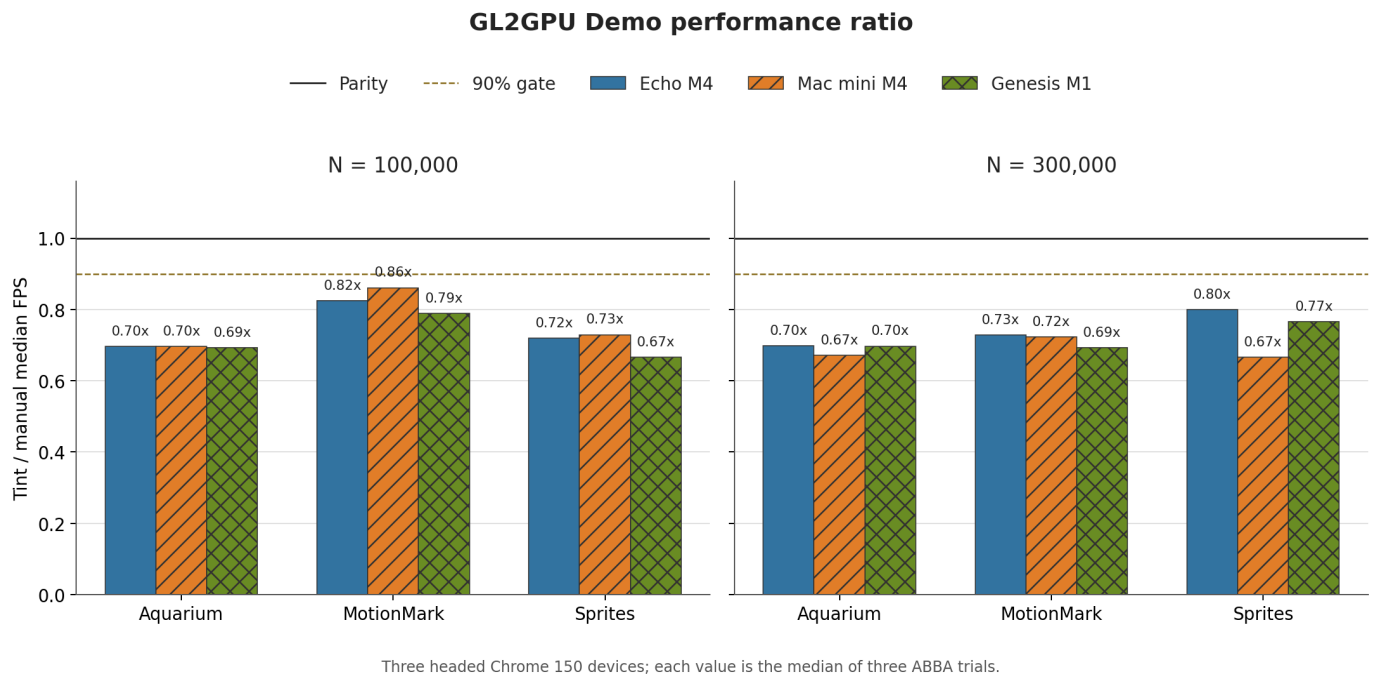
Remote GL2GPU Tint rendering (Spark v2.1.0)



7. 与 manual GL2GPU 的关系

本轮 Spark 没有 manual shader oracle，因此不能从 Spark 表直接回答“TINT WGSL 是否 已经追平手写 WGSL”。同一批机器在 2026-07-14 对 GL2GPU Demo 的 headed Chrome 150 结果提供了独立背景：

Workload	Aquarium Tint/manual	MotionMark Tint/manual	Sprites Tint/manual
100,000 objects (三机范围)	0.693x—0.697x	0.788x—0.861x	0.666x—0.729x
300,000 objects (三机范围)	0.671x—0.697x	0.693x—0.728x	0.667x—0.800x



所以最终判断分两层：

- **WebGL 语义正确性：** 本轮 runtime 修复已通过三机完整 CTS gate；Tint runtime translation 不依赖手写 shader JSON。
- **Spark 特定静态 workload：** precompute/compaction 后正确且明显快于原生 WebGL。
- **通用 Tint shader 性能：** 尚未追平 manual oracle；仍需继续做通用 WGSL/IR、 pipeline/state 与 API submit 成本分析，不能用 Spark 专用数据替代。

8. 限制与后续工作

1. 三机 Chrome 150 的 patch version 不同，小幅跨机差异不具备严格的硬件归因能力。
2. Spark 使用固定相机。相机、sorting textures 或 splat 数据变化后需要重新 precompute；本轮没有测量动态相机下的重新计算频率与 amortized cost。
3. PLY load time 包含磁盘、解析、GLSL→SPIR-V→WGSL、pipeline 和 precompute 多类成本；当前只报告端到端 load ratio，没有把它们逐项拆分。
4. Full bicycle 的性能稳定到只需 3 次，轻场景扩展到 5 次；这足以支持当前 7%—215% 的差异方向，但不适合声明 1%—3% 的微优化。
5. CTS 的两处 fixture patch 应单独向 Khronos CTS 上游报告；GL2GPU 不应实现与当前 WebGL 规范相反的 identifier 限制来迎合旧 fixture。
6. 下一轮通用性能工作应以 Aquarium、MotionMark、Sprites 的 manual shader 为 teacher，分解 Tint WGSL 的每像素 ALU、texture LOD、local store/load、pipeline variant 和 CPU submit 开销，并继续用三机画质 gate 做 A/B。

9. 结果索引

仓库内可移植、经过校验的精简数据：

- `reports/data/three-device-experiment-summary.json`
- `reports/assets/cts-triangle-three-device.png`
- `reports/assets/spark-tint-webgl-ratios.{png,svg}`
- `reports/assets/spark-remote-rendering-montage.png`
- `reports/assets/demo-tint-manual-ratios.{png,svg}`

本机未跟踪的完整原始结果：

- `output/webgl-cts-20260715-local-serial/`
- `output/multidevice-20260715/mac-mini-m4/cts-summary.json`
- `output/multidevice-20260715/genesis-m1/cts-summary.json`
- `output/multidevice-20260715/{local,mac-mini-m4,genesis-m1}/spark-requested-scenes/`
- `output/multidevice-20260714/` (GL2GPU Demo Tint/manual 背景实验)

报告数据生成器会重新验证三机 CTS 2864/2864、实际 trial 扩展规则、Spark 质量 gate、`shaderDbRequests=0` 和 vertex precompute applied 状态；任何条件不满足都会以非零状态退出，不生成新的报告数据。